

STAT 441 Predictive Risk

James Stephen

Task Definition

The objective of this project is to determine the effectiveness of using a SVM in determining financial risk. There are two situations in which we will assess the risk of an observation.

1. Predicting if a credit card transaction is fraudulent.
2. Predicting if a company is going to go bankrupt.

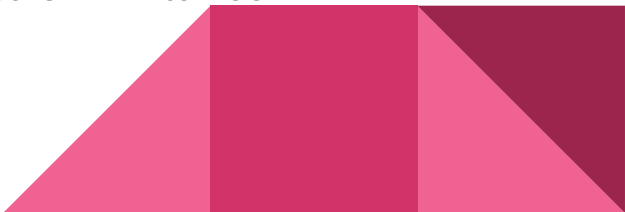


Data Description and Background

Credit Card Dataset

- 30 numerical features, 28 anonymised
- 284,807 transactions, 492 fraudulent
- “Class” is the target variable, 1 indicating fraudulent and 0 not fraudulent
- Sourced from Hugging face, data from transactions in September 2013 by European cardholders

Bankruptcy Dataset

- 95 numerical features
 - 6819 companies, 220 bankrupt
 - “Bankrupt?” is the target variable, 1 indicating bankrupt and 0 not bankrupt
 - Sourced from UCI, data from on Taiwan Economic Journal for the years 1999 to 2009
- 

Data Preprocessing

Both datasets contain no missing values

Steps:

- Train test split of 80/20
- Unbalanced data, so resample to be 1:1 between both classes
- Standardize features
- Feature selection with logistic regression with L1 penalty



Model Construction

Both models use GridSearchCV to find best parameters with a support vector machine

Parameter grid:

- $C = [0.01, 0.1, 1, 10]$
- $\text{gamma} = [\text{'scale'}, \text{'auto'}, 0.1, 1]$
- $\text{kernel} = \text{'rbf'}$

Used 5 fold cross validation for 80 fits in total

A SVM with an RBF kernel will be effective at finding optimal multidimensional margin

Model with the highest cross validation score will be our selected model

*Model assumption: All observations are independent



Function Definition and Implementation 1/3

Key libraries: Pandas, Matplotlib, Scikit-Learn

Functions:

- `load_data(filepath, target_col) => X, y`
- `downsample_training(X_train, y_train) => resampled_X, resampled_y`
- `main()`



Function Definition and Implementation 2/3

Credit Card Input Data

	Time	V1	V2	V3	V4	V5	V6	...	V23	V24	V25	V26	V27	V28	Amount
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	...	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	...	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	...	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	...	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	...	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99

Credit Card Scaled & Reduced

	0	1	2	3	4	5	6
0	0.169449	0.632472	-1.553545	0.473327	-0.563170	0.733331	-0.400120
1	0.253185	-1.915430	-1.160088	0.707359	-0.426415	0.934622	-0.171710
2	-0.886702	2.225810	-0.766889	0.661185	0.196227	0.987441	5.310939
3	-0.544586	0.216247	-0.605811	0.831933	-0.657706	0.815797	-0.320205
4	-0.574947	0.705680	-0.066971	0.890090	-0.031131	0.818647	-0.430189

Credit Card Final Results

Actual class	Probability of being class 1	Final prediction (Threshold=0.8)
0	0.671239	0
0	0.095693	0
0	0.119147	0
0	0.089536	0
0	0.412449	0

Classification Report

	precision	recall	f1-score	support
0	1.00	0.99	1.00	56853
1	0.15	0.84	0.26	109
accuracy			0.99	56962
macro avg	0.58	0.92	0.63	56962
weighted avg	1.00	0.99	0.99	56962

Function Definition and Implementation 3/3

Bankruptcy Input Data

	ROA(C) before interest and depreciation	before interest	ROA(A) before interest and % after tax	...	Net Income Flag	Equity to Liability
0		0.370594	0.424389	...	1	0.016469
1		0.464291	0.538214	...	1	0.020794
2		0.426071	0.499019	...	1	0.016474
3		0.399844	0.451265	...	1	0.023982
4		0.465022	0.538432	...	1	0.035490

Classification Report

	precision	recall	f1-score	support
0	1.00	0.85	0.92	1326
1	0.15	0.92	0.26	38
accuracy			0.85	1364
macro avg	0.57	0.89	0.59	1364
weighted avg	0.97	0.85	0.90	1364

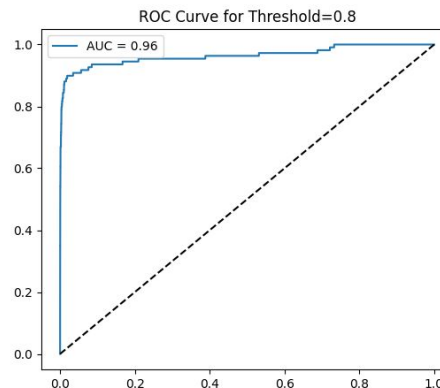
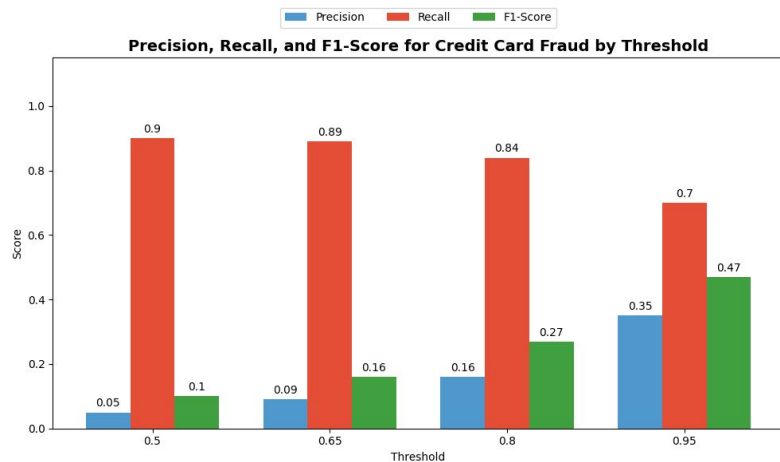
Bankruptcy Scaled & Reduced

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	-0.060390	-0.380548	-0.187110	0.722383	-0.0632	-0.095846	0.095846	-0.278594	-0.080631	-0.525809	0.730046	1.286185	-0.144535	-0.879218
1	0.363686	-0.723698	-0.231326	0.262030	-0.0632	0.817989	-0.817989	-0.774174	-0.080631	-0.384152	-1.071012	-0.715602	-0.144535	-0.451067
2	0.612499	1.247731	0.593301	1.096587	-0.0632	-0.612840	0.612840	0.542211	-0.080631	-0.525809	0.251863	-0.129040	-0.144535	-0.725888
3	1.218157	2.785850	1.685435	-0.063716	-0.0632	-1.269614	1.269614	0.712567	-0.080631	-0.525809	1.319623	0.958762	-0.144535	-0.879218
4	0.038446	-0.046818	0.089240	-1.399006	-0.0632	-1.805277	1.805277	-0.820635	-0.080631	-0.525809	1.476618	3.593034	-0.144535	-0.879218

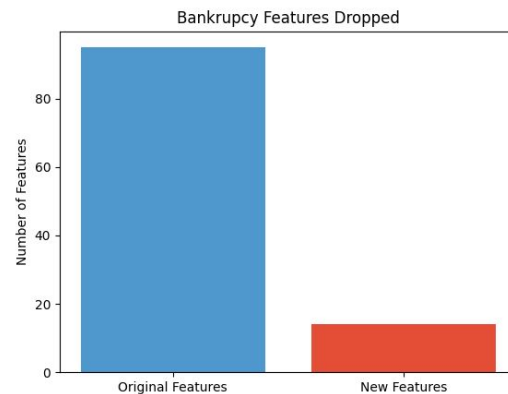
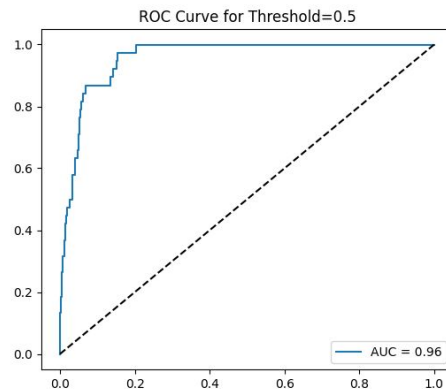
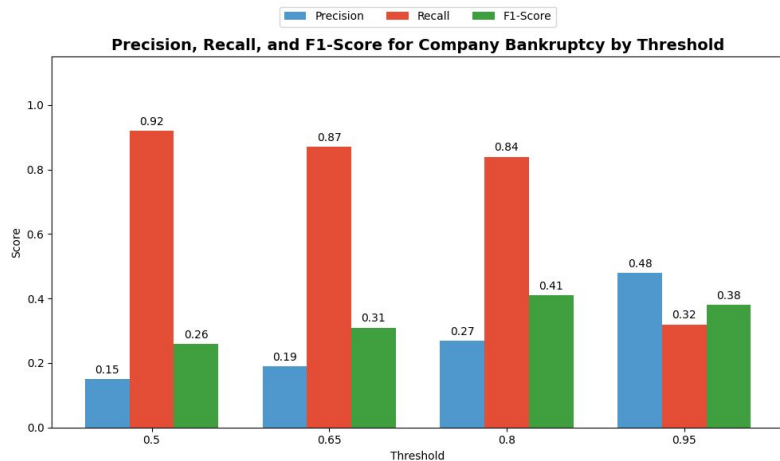
Bankruptcy Final Results

Actual class	Probability of being class 1	Final prediction (Threshold=0.5)
0	0.878800	1
0	0.109737	0
0	0.096593	0
0	0.014277	0
0	0.178580	0

Data Visualization (Credit Card Dataset)



Data Visualization (Bankruptcy Dataset)



Data Analysis and Conclusion

- Dimensionality reduction
- High Recall
- Improvement could be to test custom thresholds inside GridsearchCV
- If all 95 features were needed, would be computationally expensive
- A different default custom threshold of 50% CAN be more efficient. It's context dependent!

